

# MT – 1008 Multivariable Calculus

---

Semester Project Spring-26

Section: BCS (C, D, F)

## Mathematical Foundations of a 3D Game Engine

Multivariable calculus is the mathematical engine behind modern 3D game development. Concepts such as gradients, triple integrals, line integrals, and the Divergence Theorem directly underline how game engines handle terrain shading, rigid-body physics, particle systems, and volumetric lighting.

In this project, you are required to build the mathematical backend for a simplified 3D game engine using Python (SymPy, SciPy, NumPy, Matplotlib, etc.). You may use Python installed locally on your system to generate plots, or use online platforms such as Google Colab, Jupyter Notebook, etc. Each task connects a core calculus concept to a practical game-engine system. This will give you direct exposure to why rigorous mathematical modeling matters in real-world software development. After familiarizing yourself with the project context, you are required to perform the following tasks, compute and visualize the results using Python, and interpret your findings in meaningful real-world terms.

### The Universal Game Asset Field

To simulate a cohesive game environment, every system in this project operates on a single, unified scalar field equation that represents the core 'energy' or 'density' distribution of the game world. No other equations are to be used.

The Universal Game Asset Field is defined as:

$$E(x, y, z) = x^2y + 2y^2z + 3z^2x$$

### 1. Terrain Normal Mapping & Illumination (Gradients)

The game's terrain boundary is defined by a level surface of the universal field. Compute the gradient vector  $\nabla E$  analytically. Evaluate it at point  $P(1, 1, 1)$  on the level surface  $E(x, y, z) = 6$  to obtain the outward surface normal. Assuming a uniform directional light source vector  $L = \langle 1, 1, 1 \rangle$ , calculate the Lambertian reflectance (the dot product of the normalized gradient and the normalized light vector) at  $P$ . Using Matplotlib, render the surface boundary near  $P$  over a grid and overlay the normal vector at  $P$  as

an arrow. Plot a heatmap of the gradient magnitudes across the evaluated grid to show which regions appear brightest. Explain how this controls surface brightness in a renderer.

## 2. Physics Engine – Center of Mass (Triple Integrals)

A 3D game asset occupies a cubic bounding volume defined by  $[0, 2] \times [0, 2] \times [0, 2]$ . Its mass density distribution is dictated directly by the universal field, such that  $\delta(x, y, z) = E(x, y, z)$ . Manually set up and evaluate the triple integral for the total mass of the object enclosed within the bounding volume. Set up the integrals for the exact  $(x, y, z)$  coordinates of the center of mass.

Use a Python script (`scipy.integrate.tplquad` or equivalent) to numerically evaluate and verify your manual computations for the mass and center of mass coordinates. Plot the 3D bounding region using Matplotlib, marking the computed center of mass clearly. Explain what this tells the physics engine about how forces and torques should be applied to this asset.

## 3. Particle Systems & Energy Transfer (Vector Fields & Line Integrals)

Gravity well in the game world is modeled by taking the gradient of the universal field, creating the force field  $F = \nabla E$ . Determine whether  $F$  is a conservative vector field by computing its curl. By recognizing that the potential function  $\varphi(x, y, z)$  is exactly the universal field  $E(x, y, z)$ , use the Fundamental Theorem of Line Integrals to evaluate the total energy transferred  $\int_C F \cdot dr$  as a spaceship travels along a straight line path from the origin  $(0,0,0)$  to the point  $(2,2,2)$ .

Using NumPy and Matplotlib, plot the spaceship's 3D flight path and overlay a 3D quiver plot of the vector field  $F$  sampled on a grid around the path. Interpret whether the spaceship gains or loses energy and discuss what this implies for gameplay balancing.

## 4. Volumetric Lighting (Divergence Theorem)

A magic artifact emits a light-energy field identical to the gravity well field  $F = \nabla E$ , contained within the same cubic bounding box  $[0, 2] \times [0, 2] \times [0, 2]$ . Compute the divergence  $\nabla \cdot F$  (this is the Laplacian of the universal field). Apply the Divergence Theorem to evaluate the total outward flux of light energy across the surface of the bounding box.

Verify the total flux result numerically using `scipy.integrate`. Using Matplotlib, generate a cross-sectional heatmap at  $z = 1$  showing the divergence field across the  $xy$ -plane to visualize where light energy is being emitted most intensely. Interpret the total outward flux physically.

## Report Requirements

Students are required to submit a complete project report prepared in MS Word in their own words. A report is written in the third person; I, We, and Us are not used. The headings of the report must match the task headings listed above. The report must also include an Introduction section describing the project context, objectives, and methodology, along with a flowchart explaining the overall approach. Do not merely paste results or screenshots; meaningfully analyze and interpret every result. The report must also include a Conclusions section and a Contributions section. Provide all references at the end and attach complete Python code as an annexure. Each page must be numbered. The entire report must be typeset in Times New Roman only. Body text is 12 pt; first-level headings are 14 pt bold and second-level headings are 12 pt bold. All body text must be justified; figures and tables must be center-aligned with descriptive captions. Line and paragraph spacing must be set to 1.5. Utilize available page space efficiently and do not leave unnecessary blank space on any page.

## Guidelines

- This is a group project and carries 70 marks.
- A group may have a maximum of 3 students. One aim of this project is to enable students to work effectively as a team. Therefore, this project cannot be completed individually, though it may be done in pairs. Distributing Python scripting and mathematical proofs across your group is highly recommended.
- Plagiarized work (from the internet, AI tools, or fellow students) will result in zero marks. A similarity index of less than 18% and AI-generated content of less than 15% is permitted.
- Submit to Google Classroom as one MS Word file and one PDF of the same file, both including all codes and figures. Do not submit in .zip or .rar format.
- The file name must follow the format: ID1\_ID2\_ID3\_MT1008\_Project\_Section.
- Do not submit via email; such submissions will not be considered. Late submissions will not be considered.